

TAGS: UBER, PAYPAL

## LC 851: Loud And Rich

**Problem:** For every person  $i$  we need to return among everyone who is richer than  $i$ , the person whose quiet value is least.

### Approach 1: DFS + Memoization

If we know answer of every richer neighbour then answer for current person can be found, memoized such that every node is solved once.

```

class Solution {
public:
    void dfs(int curr, vector<vector<int>>& adj, vector<int>& ans, vector<int>& quiet) {
        if(ans[curr] != -1) {
            return; //memoization
        }

        ans[curr] = curr; //sabke hisaab se before asking chain
                        //quietest wo khud he

        for(auto &it : adj[curr]) {
            dfs(it, adj, ans, quiet);
            if(quiet[ans[it]] < quiet[ans[curr]]) {
                ans[curr] = ans[it];
            }
        }
    }

    vector<int> loudAndRich(vector<vector<int>>& richer, vector<int>& quiet) {
        int n = quiet.size();
        vector<vector<int>> adj(n);

        for(auto &it : richer) {
            int u = it[0];
            int v = it[1]; //u richer than v
            adj[v].push_back(u); //poor se rich travel kr rhe
        }

        vector<int> ans(n, -1);
        for(int i = 0; i < n; i++) { //har gareeb apne se ameer se puchega
            //ki uski chain me quietest kon

            dfs(i, adj, ans, quiet);
        }

        return ans;
    }
};

```

## Approach 2: Kahn + DP

Go from Rich to poor, initially everyone thinks they have least quiet value and they try to assert dominance among poor.

```
if quiet[ans[rich]] < quiet[ans[poor]]  
ans[poor] = ans[rich]
```

```

class Solution {
public:
    vector<int> loudAndRich(vector<vector<int>>& richer, vector<int>& quiet) {
        int n = quiet.size();
        vector<vector<int>> adj(n);
        vector<int> indeg(n, 0);

        for(auto &it : richer) {
            int u = it[0];
            int v = it[1]; //u richer than v
            adj[u].push_back(v); //rich se poor travel kr rhe
            indeg[v]++;
        }

        vector<int> ans(n);
        queue<int> q;

        for(int i = 0; i < n; i++) {
            ans[i] = i; //initially sab ko lagega main quietest hu
            if(indeg[i] == 0) {
                q.push(i);
            }
        }

        while(!q.empty()) {
            int curr = q.front();
            q.pop();

            for(auto &it : adj[curr]) {
                //rich will try to assert dominance
                if(quiet[ans[curr]] < quiet[ans[it]]) {
                    ans[it] = ans[curr];
                }

                indeg[it]--;
                if(indeg[it] == 0) {
                    q.push(it);
                }
            }
        }

        return ans;
    }
};

```